

OLS Algorithms in Deep Learning Training

A conversation exploring how the olsvered closed-form OLS algorithms
can be applied to deep learning architectures

Based on: *Solving The Ordinary Least Squares in Closed Form, Without Inversion or Normalization*
Vered Senderovich Madar & Sandra L. Batista -- arXiv:2301.01854

March 2026

1. Can OLS Solving Algorithms Be Used in Training Deep Learning?

Short answer: Yes -- for specific, well-defined sub-problems within DL architectures.

1.1 Where OLS Appears Naturally in Deep Learning

DL Scenario	OLS Role
Linear output heads	Solve for last-layer weights with frozen backbone
Linear probing	One-shot closed-form solve for transfer learning evaluation
Extreme Learning Machines	Random hidden layers; OLS for output weights
Echo State Networks	Readout weight training = pure ridge regression
Meta-learning (few-shot)	Tiny per-task OLS problem (e.g. 5-shot x 5-way = 25 samples)
NTK regime	Infinite-width DL reduces to kernel ridge regression

1.2 How the 3 olsvered Algorithms Map

Algorithm	Best DL Use Case
Alg 1 -- Modified Cholesky	Last-layer optimization, linear probing, ELM readouts, meta-learning base learner
Alg 2 -- SGSO (unnorm. GS)	Weight orthogonalization, feature decorrelation, continual learning, orthogonal init
Alg 3 -- Weighted Gen. Inverse	Importance-weighted training, curriculum learning, domain adaptation, heteroscedastic regression

1.3 Most Promising Research Direction

Alternating Closed-Form / Gradient Optimization (Galashov et al., 2025): every K gradient steps, recompute last-layer weights via Algorithm 1 in closed form. Accumulate $X^T X$ and $X^T y$ as running statistics across mini-batches. Extend with Algorithm 3 for sample-weighted variants (curriculum, class imbalance).

1.4 Key Limitations

Limitation	Notes
Non-linearity	OLS only applies to linear sub-problems -- fundamental boundary
Scale	Gram matrix ($p \times p$): practical up to $p=8192$ on CPU
Mini-batch staleness	Features change as backbone updates; accumulated stats go stale
No L1 regularization	Lasso has no closed form; ridge ($X^T X + \lambda I$) is trivial
Condition number	$\text{cond}(X^T X) = \text{cond}(X)^2$ -- use float64

2. Gram Matrix Scalability Explained

2.1 What Is the Gram Matrix?

In OLS you solve $\beta = (X^T X)^{-1} X^T y$. The term $X^T X$ is the **Gram matrix**. If X has shape (n, p) then $X^T X$ has shape $(p \times p)$, regardless of the number of samples.

2.2 Why p Matters More Than n

In deep learning, p is the feature/hidden dimension of the layer being solved:

Architecture	Typical p	Gram Matrix	Memory (float64)
Small CNN head	512	512 x 512	2 MB
ResNet-50	2,048	2,048 x 2,048	32 MB
ViT-Large	1,024	1,024 x 1,024	8 MB
GPT-2 medium	4,096	4,096 x 4,096	128 MB
LLaMA-7B	4,096	4,096 x 4,096	128 MB
LLaMA-65B	8,192	8,192 x 8,192	512 MB
GPT-4 class	~12,288+	12,288 x 12,288	1.1 GB
Large MoE	65,536	65,536 x 65,536	32 GB

2.3 The $O(p^3)$ Bottleneck

LU decomposition costs $O(p^3)$ FLOPs. Double p and compute grows 8x:

p	FLOPs	Wall Time (approx)
512	~134 million	milliseconds
2,048	~8.6 billion	~1 second
8,192	~550 billion	~30-60 seconds
65,536	~281 trillion	hours on CPU

2.4 Practical Boundaries

Range	Assessment
$p \leq 2,048$	Instant -- use freely (covers most vision models)
$p \leq 4,096$	Fast -- use every few gradient steps (covers most LLMs)
$p \leq 8,192$	Feasible -- use sparingly (large LLMs)
$p > 8,192$	Switch to iterative methods (CG, truncated SVD, sketching)

3. Gram Matrix Usage by Algorithm

Algorithm	Uses Gram Matrix?	What It Computes	Shape
Alg 1 -- Modified Cholesky	Yes	$G = [X y]^T[X y]$ (augmented)	$(p+1) \times (p+1)$
Alg 2 -- SGSO	No	Column-wise dot products on X directly	operates on (n, p)
Alg 3 -- Weighted Gen. Inverse	Yes	$X^T W X$ (weighted Gram)	$p \times p$

3.1 Algorithm 1 (line 75 in algorithms.rs)

```
let gram = xy.transpose() * &xy;    // (p+1) x (p+1)
```

Forms the augmented Gram matrix $[X|y]^T[X|y]$, then LU-decomposes it. Bounded by $O(p^3)$.

3.2 Algorithm 2 (lines 186-202)

```
let num = qj.dot(&qi);    // scalar dot product
let den = qi.dot(&qi);    // scalar dot product
```

No Gram matrix at all. Works column-by-column with dot products directly on the (n, p) matrix. Cost is $O(np^2)$ -- scales with the number of samples.

3.3 Algorithm 3 (line 243)

```
let xtwx = &xtw * x;    // p x p
```

Forms the weighted Gram matrix $X^T W X$, then solves via LU. Same $O(p^3)$ bound as Algorithm 1.

Bottom line: The $p=8,192$ wall applies to **Algorithms 1 and 3** only. Algorithm 2 (SGSO) has different scaling -- $O(np^2)$.

4. What Do n and p Stand For?

Symbol	Meaning	Example
n	Number of observations (rows / samples / data points)	1,000 patients in a study
p	Number of predictors (columns / features / variables)	10 measurements per patient

The design matrix X has shape **(n x p)**. In deep learning terms: n = batch/dataset size, p = hidden/feature dimension of a layer.

5. When Can Closed-Form OLS Replace Gradient Descent?

5.1 The One Rule

If the layer is linear and the loss is squared error, you can replace gradient descent with a direct solve. Gradient descent *iterates* toward the answer. OLS *computes* it in one shot.

5.2 Where It Can Replace GD Today

Scenario	Why OLS Works	GD Steps Replaced
Last linear layer (frozen backbone)	It is literally $y = X\beta$	Hundreds/thousands of SGD steps -> 1 solve
Linear probing	Frozen features, linear head	Entire training loop -> 1 solve
ELM / Reservoir readout	Random fixed layers, only readout trained	All training -> 1 solve
Few-shot meta-learning	Tiny linear problem per task	Per-task fine-tuning -> instant solve

5.3 The Practical Payoff

Metric	SGD on Last Layer	OLS Solve
Steps to converge	100 -- 10,000	1
Result	Approximate	Exact (to machine precision)
Hyperparameters	LR, momentum, schedule	None
When $p \leq 4096$	Minutes of training	Milliseconds

5.4 Where It Cannot Replace GD

Situation	Why Not
Non-linear layers (ReLU, attention)	No closed-form solution; loss landscape is non-convex
Cross-entropy loss	OLS requires squared error; classification uses log-loss
Regularization beyond L2	Dropout, L1, batch norm have no closed form
$n < p$ (more features than samples)	Gram matrix is singular; need ridge regression
Streaming / online learning	OLS needs all data at once (mitigated by incremental $X^T X$)

6. Where Do These Algorithms Unlock OLS Due to Better Efficiency?

6.1 Gram + LU vs Standard SVD

Standard `numpy.linalg.lstsq` uses SVD at $O(np^2)$ on the full (n, p) matrix. Algorithms 1 & 3 first compress to the (p, p) Gram matrix then solve via LU at $O(np + p^3)$:

n (samples)	p (features)	SVD Cost	Gram + LU Cost	Speedup
10,000	64	41M	0.9M	~45x
100,000	128	1.6B	15M	~107x
1,000,000	256	65B	273M	~238x
10,000	2,048	42B	29B	~1.4x

Sweet spot: $n \gg p$ (many samples, moderate feature dimension). The bigger the ratio n/p , the larger the advantage.

6.2 Algorithm 1 -- OLS Inside Training Loops

With incremental Gram accumulation, the entire dataset is compressed into a $(p \times p)$ matrix. Each solve is only $O(p^3)$, independent of total n . If each solve takes 50ms instead of 500ms, you can run it 10x more frequently.

```
# Running Gram matrix -- O(batch_size * p) per step
G += X_batch.T @ X_batch      # accumulate (p x p)
h += X_batch.T @ y_batch      # accumulate (p,)

# Every K steps -- solve is only O(p**3)
beta = lu_solve(G, h)        # Algorithm 1 core
```

6.3 Algorithm 2 -- Differentiable Orthogonalization Layer (Unique Unlock)

Standard Gram-Schmidt has square roots in normalization. Backpropagating through $1/\sqrt{x}$ gives $d/dx = -1/(2x^{3/2})$, which **explodes when x approaches 0**.

Algorithm 2 (SGSO) has no square roots. Its backward pass only involves divisions by dot products $q_i^T q_i$, which are more numerically stable. This means it can be used as a differentiable layer inside a neural network with stable gradients.

Use cases this unlocks:

- Orthogonal weight constraints as a cheap forward-pass operation
- Feature decorrelation layers that are differentiable and stable
- Continual learning: project gradients onto orthogonal complements during training

6.4 Algorithm 3 -- Attention-Weighted Readouts (Practical Unlock)

When an attention mechanism produces per-sample weights, Algorithm 3 computes the weighted-optimal linear projection via LU solve **without explicit matrix inversion**. This makes weighted OLS cheap and safe enough to run per-attention-head, per-layer.

6.5 Summary: Unique Edge per Algorithm

Algorithm	Unique Efficiency Advantage	What It Unlocks
Alg 1 -- Modified Cholesky	$O(np + p^3)$ vs $O(np^2)$; incremental Gram accumulation	OLS in training inner loops, frequent last-layer updates
Alg 2 -- SGSO	No sqrt -> stable gradients; non-iterative	Differentiable orthogonalization as a network layer
Alg 3 -- Weighted GI	Weighted OLS via LU (no explicit inversion)	Per-head attention-weighted readouts; stable weighted solves

Overarching theme: these algorithms make OLS cheap enough and stable enough to run *inside* a forward/backward pass, not just as a one-time offline computation.

References

- [1] Senderovich Madar, V. & Batista, S. (2023). Solving The Ordinary Least Squares in Closed Form, Without Inversion or Normalization. arXiv:2301.01854.
- [2] Galashov, A. et al. (2025). Closed-Form Last Layer Optimization. arXiv:2510.04606.
- [3] Bertinetto, L. et al. (2019). Meta-Learning with Differentiable Closed-Form Solvers. ICLR 2019.
- [4] Kumar, A. et al. (2024). Understanding Linear Probing then Fine-tuning. NeurIPS 2024.
- [5] Non-iterative CNN Training via Gram-Schmidt Process. Neural Processing Letters, 2025.
- [6] M-estimation ELM Ensembles. Nature Scientific Reports, 2025.
- [7] Monga, V. et al. (2019). Algorithm Unrolling: Interpretable, Efficient Deep Learning. arXiv:1912.10557.
- [8] Overcoming Catastrophic Interference Using Gram-Schmidt Orthogonalization. PLOS ONE, 2014.